Optimizing Distributed Matrix Multiplication on the Cerebras WSE-2

Andrew Joseph Kim
ajk999@cornell.edu
Cornell University
Ithaca, NY, USA

Abstract

We optimize a distributed $C{=}A{\times}B$ kernel on a Cerebras WSE-2 8×8 PE grid (M=128, H=256, N=64; per-PE tiles 16×32, 32×8). We characterize the per-`@fmacs` hardware floor at 40 cy (SRAM single-read-port: 2×16+7 pipeline cycles), disprove SRAM bank conflicts as a bottleneck, and show that comptime loop unrolling causes 31–35 K cycle regressions from instruction-cache pressure. Our best implementation uses h-outer GEMV with runtime loop bounds and batch receive, achieving **92 K cycles** against a theoretical floor of ≈82 K.

CCS Concepts

• **Computer systems organization → Parallel architectures**.

Keywords

Cerebras WSE-2, matrix multiplication, GEMV, CSL, wafer-scale

ACM Reference Format:
Andrew Joseph Kim. 2025. Optimizing Distributed Matrix Multiplication on the Cerebras WSE-2. In *Proceedings of CS5220: Applications of Parallel Computers (CS5220 '25).* ACM, New York, NY, USA, 1 page. https://doi.org/XXXXXXX.XXXXXXX

1 Problem and Architecture

Each of the 64 PEs owns $A[16{\times}32]$, $B[32{\times}8]$, $C[16{\times}8]$ in column-major SRAM. To compute C, each PE must accumulate B-column contributions from all 8 y-neighbors via a relay broadcast (PEs send north to y=0; y=0 relays south), then reduce partial sums along x via a rotating ring. All data routing is explicit in CSL; there are no collective primitives.

2 GEMV Strategies

Column-major (original): for each of 64 output columns, copy $B[:, j]$ to `work_B` (49 cy), run 32 `@fmacs` into `red_in`, then reduce (96 cy).

H-outer (current best): outer loop over $h \in [0, 32)$, inner loop over $j \in [0, 8)$, accumulating into `accum[8][16]`. Eliminates the 49 cy copy by reading `batch_B[j*dH+h]` directly; reuses $A[:, h]$ DSD across 8 j-iterations; writes to distinct per-column accumulators (no read-after-write stall). Both loops use runtime bounds to prevent compiler unrolling.

3 Key Findings

Table 1: Tested Configurations (8×8 grid, M=128, H=256, N=64)

Configuration	Cycles
Column-major + batch recv + copy	93 K
+ SRAM bank padding (1 word)	92 K (*negligible*)
H-outer + batch recv + runtime bounds	**92 K**
Dual accumulator variant	124 K (+32 K)
H-outer + comptime bounds	128 K (+36 K)
Per-column recv + column-done task	123 K (+31 K)

@fmacs floor. Each call (extent=16) serializes two 16-element SRAM reads through a single read port: 2×16+7=39≈40 cy. Over 2048 calls this gives ≈82 K cycles of irreducible compute.
Unrolling is harmful. With `-max-inlined-iterations 150`, comptime bounds fully unroll the 32×8=256 `@fmacs` per batch into a ~8 KB instruction block that exceeds the CE instruction cache, causing repeated SRAM re-fetches (+31–36 K cycles). Runtime bounds prevent unrolling at a cost of only ≈2–5 cy per call.
Per-column dispatch is expensive. Switching from 8 batch-done task invocations to 64 column-done invocations added ≈31 K cycles from task dispatch overhead, loss of batched reduce amortization, and per-column `red_in` clearing.

4 Remaining Gap and Future Work

The 92 K result sits ≈10 K above the ≈82 K floor. The residual overhead is: runtime loop control (≈4 K), runtime DSD construction (≈2 K), and un-overlapped ring reduction (64×96≈6 K, partially pipelined). The highest-confidence path forward is *async reduce with a copy buffer*: copy `red_in` to a 16-word staging buffer (16 cy), launch async reduce from it, and immediately begin the next GEMV—hiding ≈96 cy of reduction inside each ≈1280 cy GEMV for a projected ≈5–6 K cycle saving.

Unpublished working draft. Not for distribution.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted. Unpublished notices above must be honored. Distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
CS5220 '25, Cornell University
© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-1-4503-XXXX-X/2025/05
https://doi.org/XXXXXXX.XXXXXXX

Unpublished working draft. Not for distribution.